

Process injection

Breaking All macOS Security Layers
With a Single Vulnerability

```
14
15 @implementation AppDelegate
16
17 - (void)applicationDidFinishLaunching:(NSNotification *)aNotification {
18     // Insert code here to initialize your application
19 }
20
21
22 - (void)applicationWillTerminate:(NSNotification *)aNotification {
23     // Insert code here to tear down your application
24 }
25
26
27 - (BOOL)applicationSupportsSecureRestorableState:(NSApplication *)app {
28     return YES;
29 }
30
31
32 @end
33
```



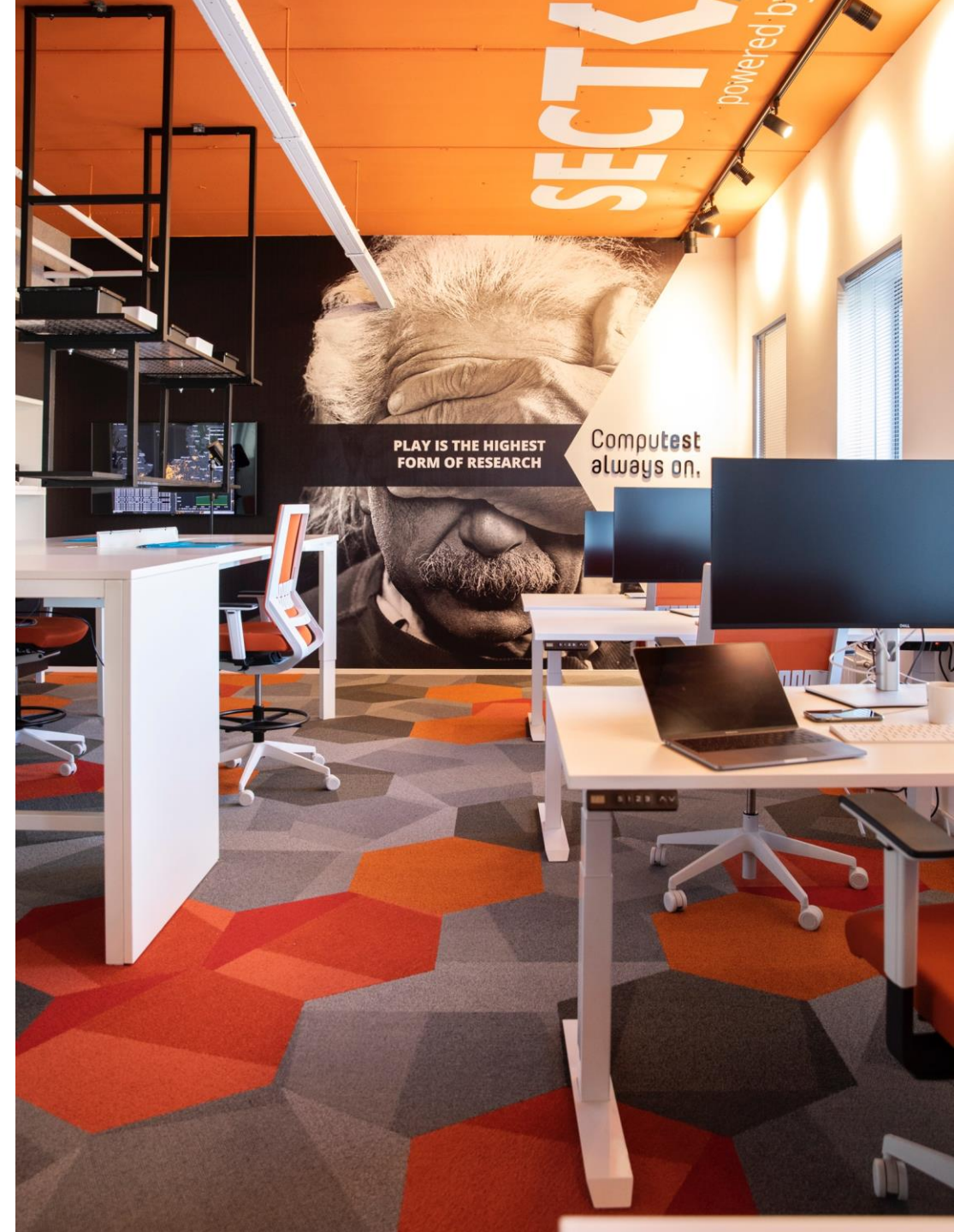
Hello!

I'm Thijs Alkemade

Security researcher at Computest

About me

- > Thijs Alkemade (@xnyhps)
- > Security researcher at Computest
- > Computest research lab: Sector 7
- > Other recent work includes:
 - 0click Zoom RCE at Pwn2Own Vancouver 2021
 - Winning Pwn2Own Miami 2022 with 5 ICS vulnerabilities



In this talk

1. macOS security model
2. CVE-2021-30873: process injection using saved states
3. Using process injection for:
 - Sandbox escape
 - Privilege escalation
 - SIP bypass

macOS security model

In macOS 12 Monterey

Old *NIX security model

- > Users are security boundaries, processes are not
- > File permissions: POSIX flags
- > Attach debugger: target must run as same user
- > root has full access

```
-rw-r--r--  1 talkemade  wheel  0 Aug 11 12:57 file
```


System Integrity Protection

NEW

Security policy applying to every process, including privileged code running unsandboxed

Extends additional protections to system components on disk and at runtime

System binaries can only be modified by Apple Installer and Software Update, and no longer permit runtime attachment or code injection



SIP restrictions

- > "Dangerous" operations require **entitlements**:
 - Loading a kernel extension
 - Modifying system files
 - Debugging system processes
- > More and more restrictions in each macOS release
 - Debugging any app is now restricted
 - "Data vaults" with restricted file access

```
$ ls ~/Library/Mail/
```

```
ls: /Users/talkemade/Library/Mail/: Operation not permitted
```

```
$ sudo ls ~/Library/Mail/
```

```
ls: /Users/talkemade/Library/Mail/: Operation not permitted
```

```
$ █
```

```
$ codesign -dvvv --entitlements - /System/Applications/Mail.app/  
Executable=/System/Applications/Mail.app/Contents/MacOS/Mail  
Identifier=com.apple.mail  
Format=app bundle with Mach-O universal (x86_64 arm64e)  
[...]  
    [Key] com.apple.rootless.storage.Mail  
    [Value]  
        [Bool] true
```

Process injection

- > Process A executing code "as" process B
- > Many techniques are restricted by SIP
- > Hardened runtime prevents it in apps:
 - No DYLD_* environment variables
 - Library validation
- > But macOS is old, and large...

Platform Policy

Restricted processes

NEW

task_for_pid() / processor_set_tasks() fail with EPERM

Mach special ports are reset on exec(2)

dyld environment variables are ignored

dtrace probes unavailable

```
$> sudo lldb -n Finder
(lldb) process attach --name Finder
error: attach failed: attach failed: lost connection
```

Process injection

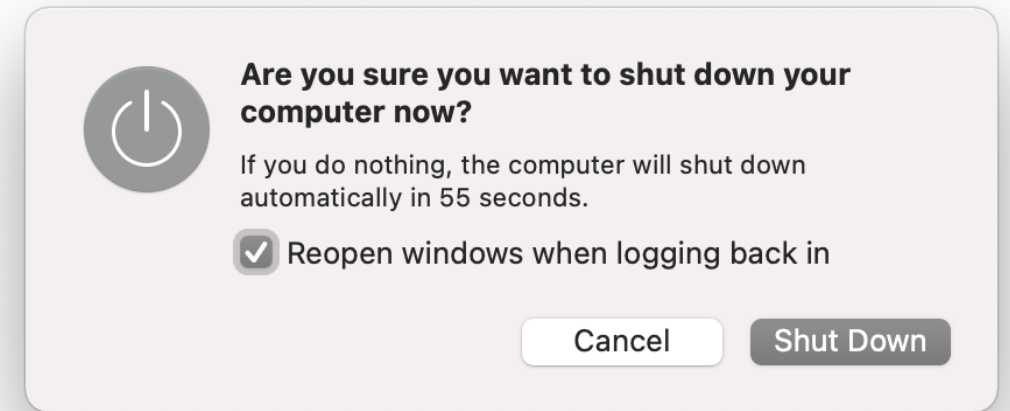
- > Common in third-party apps
- > Abuse TCC permissions: access webcam, microphone, etc.
- > Downgrade attacks often work
- > What's better than process injection in one app? Process injection everywhere!

CVE-2021-30873

Process injection in AppKit

Saved state feature

- > Re-opening the windows of an app when relaunched
- > Restores unsaved documents
- > Works automatically, can be extended by developers



Saved state storage

> Stored in:

- ~/Library/Saved Application State/<ID>.savedState

> windows.plist

- array of all windows, each with an encryption key

> data.data

- custom format, AES-CBC encrypted serialized object per record

```
00000000 4e 53 43 52 31 30 30 30 00 00 00 01 00 00 01 b0 |NSCR1000.....|
00000010 ec f2 26 b9 8b 06 c8 d0 41 5d 73 7a 0e cc 59 74 |..&.....A]sz..Yt|
00000020 89 ac 3d b3 b6 7a ab 1b bb f7 84 0c 05 57 4d 70 |..=..z.....WMp|
00000030 cb 55 7f ee 71 f8 8b bb d4 fd b0 c6 28 14 78 23 |.U..q.....(.x#|
00000040 ed 89 30 29 92 8c 80 bf 47 75 28 50 d7 1c 9a 8a |..0)....Gu(P....|
00000050 94 b4 d1 c1 5d 9e 1a e0 46 62 f5 16 76 f5 6f df |....]...Fb..v.o.|
00000060 43 a5 fa 7a dd d3 2f 25 43 04 ba e2 7c 59 f9 e8 |C..z../%C...|Y..|
00000070 a4 0e 11 5d 8e 86 16 f0 c5 1d ac fb 5c 71 fd 9d |....].....\q..|
00000080 81 90 c8 e7 2d 53 75 43 6d eb b6 aa c7 15 8b 1a |....-SuCm.....|
00000090 9c 58 8f 19 02 1a 73 99 ed 66 d1 91 8a 84 32 7f |.X....s..f....2.|
000000a0 1f 5a 1e e8 ae b3 39 a8 cf 6b 96 ef d8 7b d1 46 |.Z....9..k...{.F|
000000b0 0c e2 97 d5 db d4 9d eb d6 13 05 7d e0 4a 89 a4 |.....}.J...|
000000c0 d0 aa 40 16 81 fc b9 a5 f5 88 2b 70 cd 1a 48 94 |..@.....+p..H.|
000000d0 47 3d 4f 92 76 3a ee 34 79 05 3f 5d 68 57 7d b0 |G=0.v:.4y.?]hW}.|
000000e0 54 6f 80 4e 5b 3d 53 2a 6d 35 a3 c9 6c 96 5f a5 |To.N[=S*m5..l._.|
000000f0 06 ec 4c d3 51 b9 15 b8 29 f0 25 48 2b 6a 74 9f |..L.Q...).%H+jt.|
00000100 1a 5b 5e f1 14 db aa 8d 13 9c ef d6 f5 53 f1 49 |.[^.....S.I|
00000110 4d 78 5a 89 79 f8 bd 68 3f 51 a2 a4 04 ee d1 45 |MxZ.y..h?Q.....E|
00000120 65 ba c4 40 ad db e3 62 55 59 9a 29 46 2e 6c 07 |e..@...bUY.)F.l.|
00000130 34 68 e9 00 89 15 37 1c ff c8 a5 d8 7c 8d b2 f0 |4h....7.....|...|
00000140 4b c3 26 f9 91 f8 c4 2d 12 4a 09 ba 26 1d 00 13 |K.&.....-J.&...|
00000150 65 ac e7 66 80 c0 e2 55 ec 9a 8e 09 cb 39 26 d4 |e..f...U.....9&|
00000160 c8 15 94 d8 2c 8b fa 79 5f 62 18 39 f0 a5 df 0b |.....,y_b.9....|
00000170 3d a4 5c bc 30 d5 2b cc 08 88 c8 49 d6 ab c0 e1 |=.\.0.+....I....|
00000180 c1 e5 41 eb 3e 2b 17 80 c4 01 64 3d 79 be 82 aa |..A.>+....d=y...|
00000190 3d 56 8d bb e5 7a ea 89 0f 4c dc 16 03 e9 2a d8 |=V...z...L....*|
000001a0 c5 3e 25 ed c2 4b 65 da 8a d9 0d d9 23 92 fd 06 |.>%..Ke.....#...|
```

Serialization vulnerabilities

- > Insecure deserialization can lead to RCE
 - Well known in C#, Java, Python, Ruby...
- > Apple's serialization is NSCoder
- > Added NSSecureCoding in 10.8 (2012)



// Insecure

```
id obj = [decoder decodeObjectForKey:@"myKey"];  
if (![obj isKindOfClass:[MyClass class]]) { /* ...fail... */ }
```

// Secure

```
id obj = [decoder decodeObjectOfClass:[MyClass class]  
         forKey:@"myKey"];
```

Exploiting for process injection

1. Create a saved state using a malicious serialized object
2. Write it to the saved state directory of the other app
3. Launch other app
4. App automatically deserializes our object
5. Execute code in the other app!

What object to write?

- > ysoserial-objective-c?
- > Google Project Zero writeups?

Insecure deserialization with NSCoder

And defeating the hardened runtime by executing Python

Search for an object chain

- > Disassemble -initWithCoder: methods
- > Surprisingly, many classes do not support secure coding!
- > ...but in most cases it only recursively decodes instance variables

Step 1: NSRuleEditor

- > NSRuleEditor creates a binding to a keypath also from the archive:

```
ID NSRuleEditor::initWithCoder:(ID param_1,SEL param_2,ID unarchiver)
{
    ...
    id arrayOwner = [unarchiver decodeObjectForKey:@"NSRuleEditorBoundArrayOwner"];
    ...
    if (arrayOwner) {
        keyPath = [unarchiver decodeObjectForKey:@"NSRuleEditorBoundArrayKeyPath"];
        [self bind:@"rows" toObject:arrayOwner withKeyPath:keyPath options:nil];
    }
    ...
}
```

- > Result: call any zero-argument method on a deserialized object

Step 2: NSCustomImageRep

> NSCustomImageRep obtains an object and selector from the archive:

```
ID NSCustomImageRep::initWithCoder:(ID param_1,SEL param_2,ID unarchiver)
{
    ...
    self.drawObject = [unarchiver decodeObjectForKey:@"NSDrawObject"];
    id drawMethod = [unarchiver decodeObjectForKey:@"NSDrawMethod"];
    self.drawMethod = NSSelectorFromString(drawMethod);
    ...
}
```

Step 2: NSCustomImageRep

> NSCustomImageRep in -draw then calls the selector on the object:

```
void ____24-[NSCustomImageRep_draw]_block_invoke(long param_1)
{
    ...
    [self.drawObject performSelector:self.drawMethod withObject:self];
    ...
}
```

> Result: call any method on a deserialized object (limited control over arguments)

Deserialization to arbitrary code execution

1. Call zero-argument methods on deserialized objects
2. Call any method on deserialized objects
3. Create objects not implementing NSCoder
4. Call zero-argument methods on arbitrary objects
5. Call any method on arbitrary objects
6. Evaluate AppleScript

Hardened runtime vs Python

> Hardened runtime:

- No JIT memory pages allowed
- No unsigned memory allowed
- Library validation (all libraries signed by same developer or Apple)
- No DYLD environment variables

> Solution:

- Load Python.framework (signed by Apple)
- import ctypes

> How to evaluate Python?

AppleScript-Objective-C bridge

- > Load AppleScriptObjC.framework
- > Can load scripts from unsigned bundles
- > Can:
 - Create Objective-C objects
 - Call methods
 - Call C-functions
 - Can **not**: create non-object pointers, structs, C-strings
- > Call Py_Main(0, NULL): reads script from stdin!

on initialize()

 tell current application to NSLog("Hello world from AppleScript!")

 current application's NSBundle's alloc's

 initWithPath_("/System/Library/Frameworks/Python.framework")'s load

 current application's Py_Initialize()

 tell current application to NSLog("Py_Main: %d",

 Py_Main(0, reference))

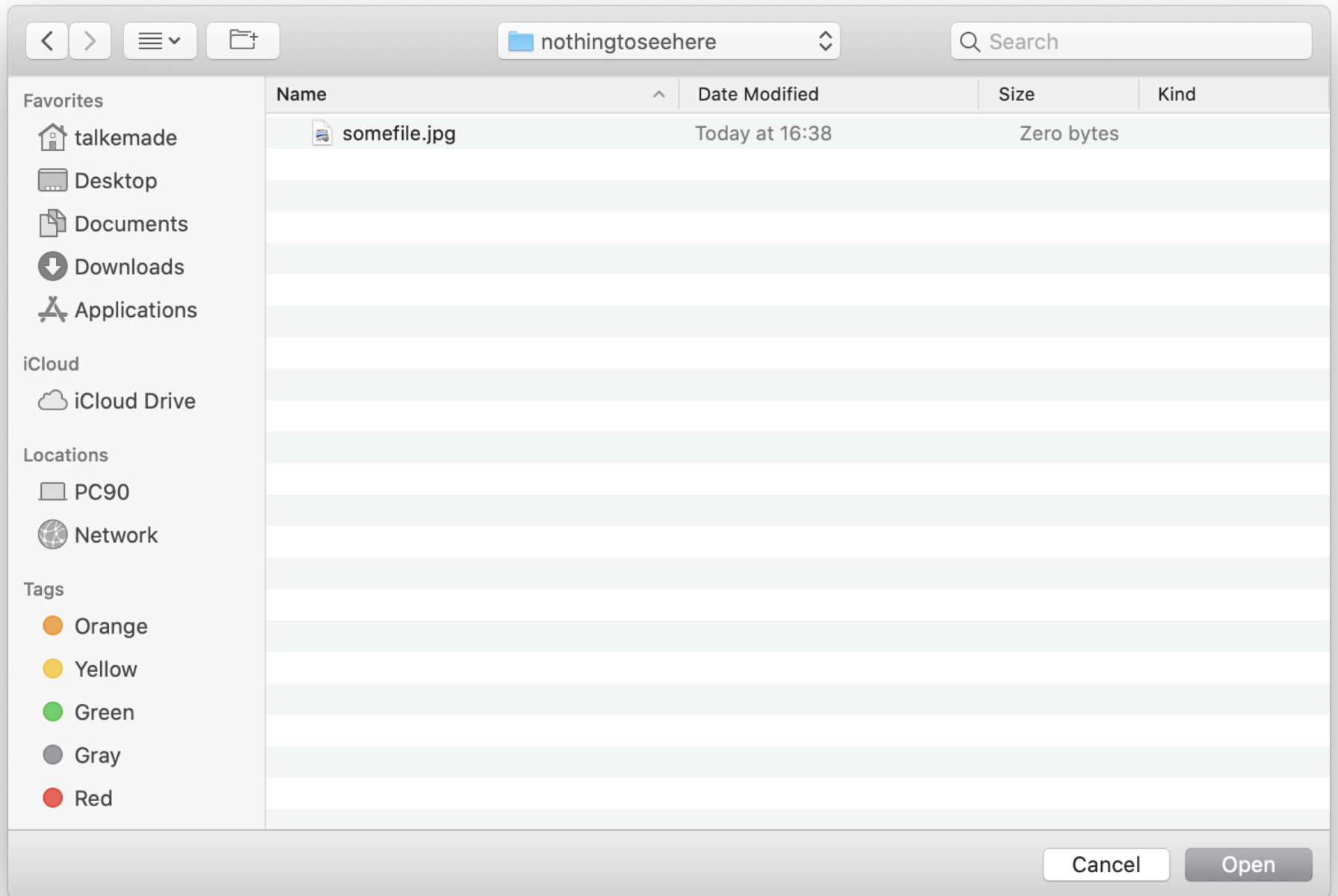
end initialize

Deserialization to arbitrary code execution

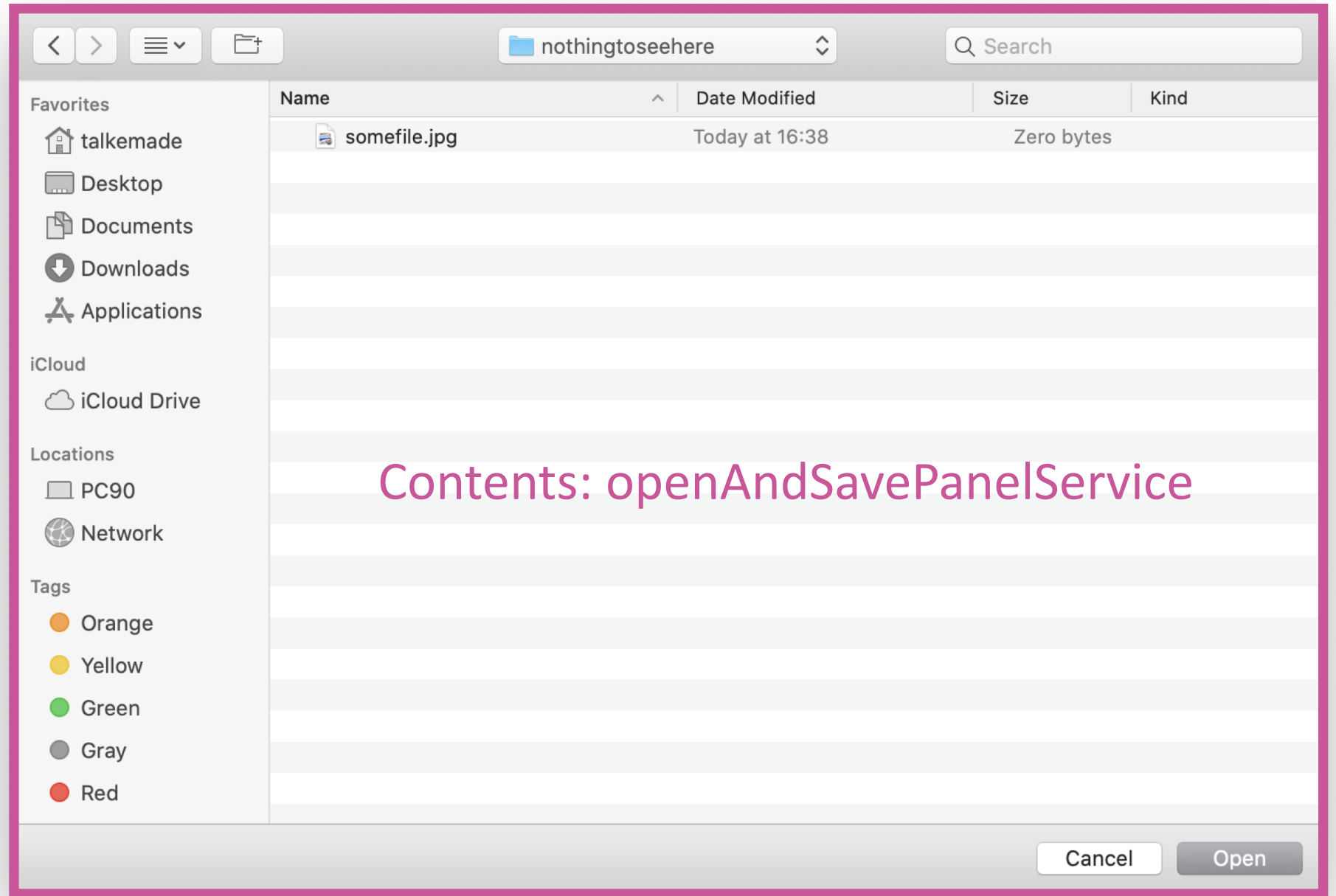
1. Call zero-argument methods on deserialized objects
2. Call any method on deserialized objects
3. Create objects not implementing NSCoder
4. Call zero-argument methods on arbitrary objects
5. Call any method on arbitrary objects
6. Evaluate AppleScript
7. Evaluate AppleScript with the AppleScript-Objective-C bridge
8. Evaluate Python
9. Import ctypes
10. Execute code equivalent to native code

Exploitation

Sandbox escape



Window: the app



Sandbox escape

- > Open/save panel loaded its saved state from the same files as the app!
 - Write new object in the app's own saved state directory
 - Open a panel
 - Sandbox escaped!
- > Fixed in 11.3: no long shares directory

CoreFoundation

Available for: macOS Big Sur

Impact: A malicious application may be able to leak sensitive user information

Description: A validation issue was addressed with improved logic.

CVE-2021-30659: Thijs Alkemade of Computest

Exploitation

Privilege escalation to root

Privilege escalation

> Use the same technique as “Unauthd - Logic bugs FTW” by Ilias Morad

> **First, find an app with entitlement:**

com.apple.private.AuthorizationServices

containing:

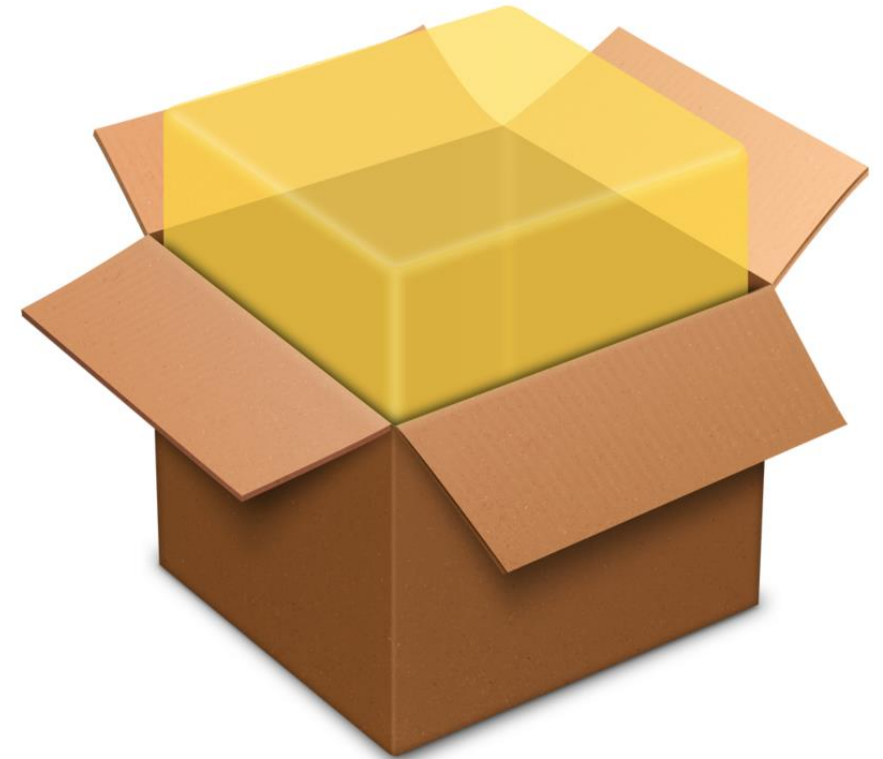
system.install.apple-software



Install Command Line Developer Tools.app

Privilege escalation

- > Then, install this package to a RAM disk
- > It runs a post-install script from the target disk as root
 - Target disk may not even have macOS!
 - Mounting a RAM disk does not require root



macOSPublicBetaAccessUtility.pkg

Installer package - 84 KB

Exploitation

SIP filesystem bypass

SIP filesystem bypass

- > App from the macOS Big Sur beta installation dmg
- > Has the entitlement:
 - com.apple.rootless.install.heritable
- > Very powerful entitlement: access all SIP protected files!
 - Heritable as a bonus, so can spawn a reverse shell

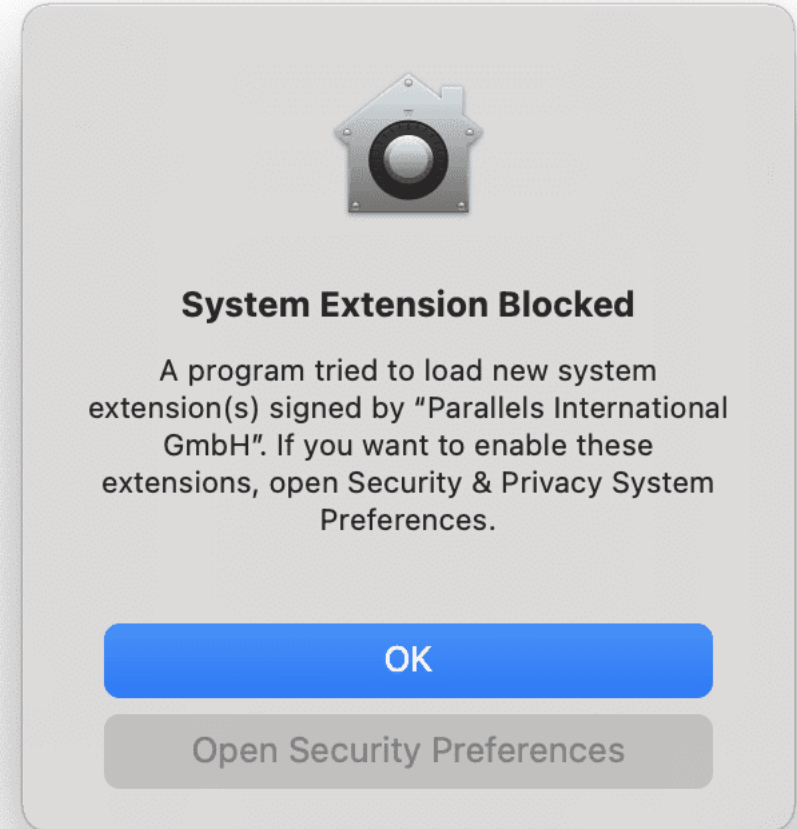


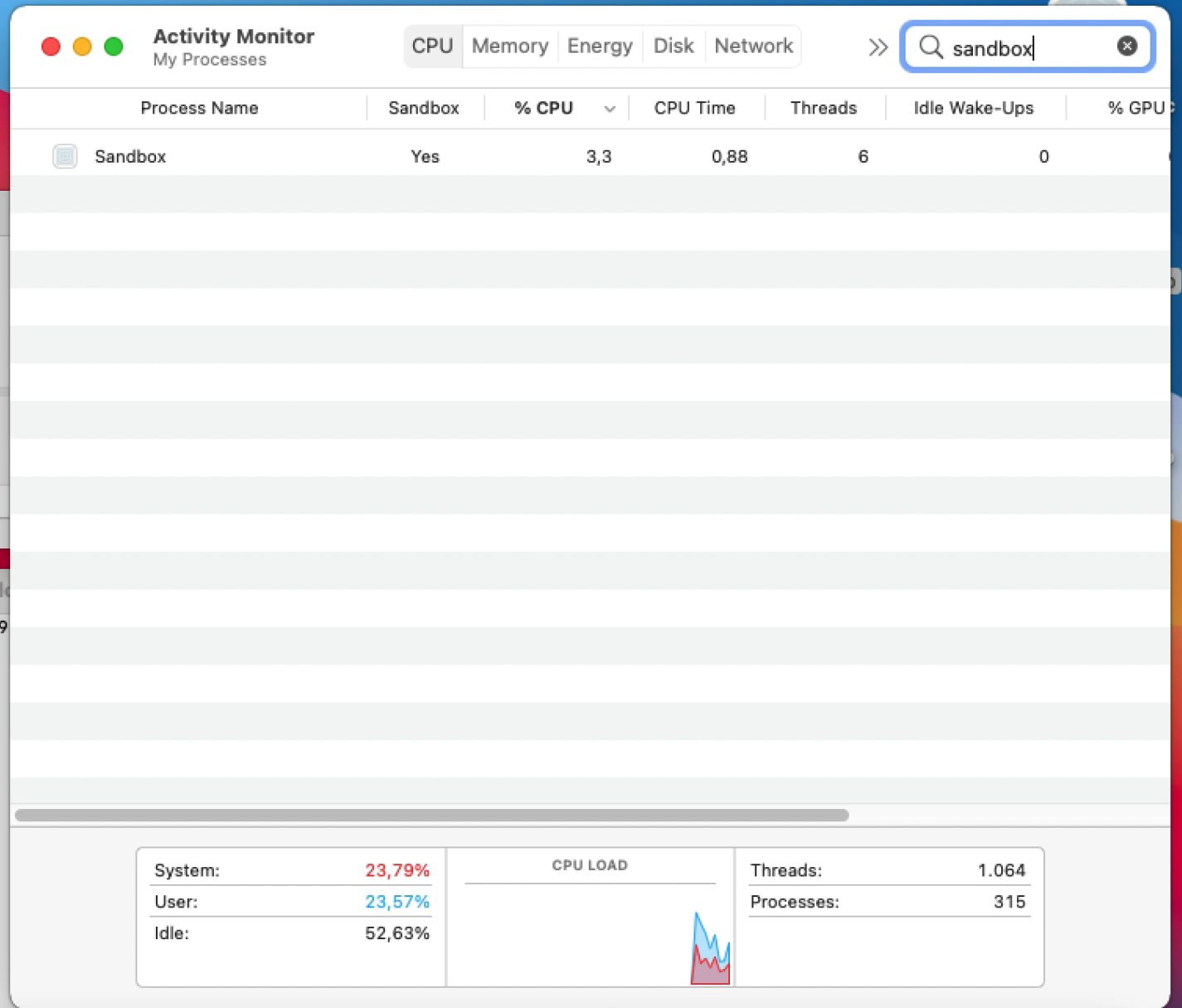
macOS Update Assistant.app

Application - 335 KB

SIP filesystem bypass: result

- > Read mail, messages, Safari history, etc. of all users
- > Grant ourselves permission for webcam, microphone, etc.
- > Powerful persistence (SIP protected locations, delete MRT)
- > Load a kernel extension without user approval





The fixes

The fixes

- > In Monterey, an app can indicate that it only accepts secure serialized objects in its saved state
 - Already enabled for Apple's apps
 - Existing apps may want to store objects that do not implement secure deserialization
 - Unclear if exploitable when apps don't use custom serialized objects
- > Reported December 4, 2020
- > Sandbox escape fixed (CVE-2021-30659) in 11.3 (April 26, 2021)
- > Fix introduced in macOS Monterey 12.0.1 (October 25, 2021)
 - Unclear if backported to Big Sur and Catalina

Conclusion

Conclusion

- > macOS has a security boundary between processes
- > Process injection vulnerabilities can be used to break those boundaries
- > CVE-2021-30873 was a process injection vulnerability affecting AppKit apps
- > We used it to escape the sandbox, privilege escalation, bypassing SIP
- > Fixed by Apple in Monterey

Takeaways

- > macOS security keeps adding more and more defensive layers
- > Adding new layers to an established system is hard
 - Code written 10+ years ago without security requirements is today's attack surface
- > Effort of attackers may not increase with more layers
 - Use the same bug for multiple layers or skip layers

References

- > <https://wojciechregula.blog/post/abusing-electron-apps-to-bypass-macos-security-controls/>
- > <https://googleprojectzero.blogspot.com/2020/01/remote-iphone-exploitation-part-1.html>
- > <https://googleprojectzero.blogspot.com/2022/03/forcedentry-sandbox-escape.html>
- > https://a2nkf.github.io/unauthd_Logic_bugs_FTW/
- > <https://mjtsai.com/blog/2015/11/08/the-java-deserialization-bug-and-nssecurecoding/>
- > <https://developer.apple.com/documentation/foundation/nssecurecoding?language=objc>
- > <https://github.com/frohoff/ysoserial>
- > <https://github.com/pwntester/ysoserial.net>